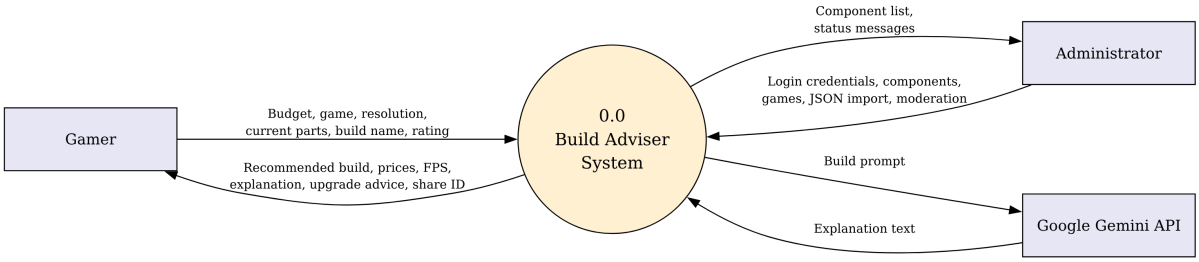
# BuildAdvisor

## DFD, E-R Diagram & Application Control Flow – Explanation

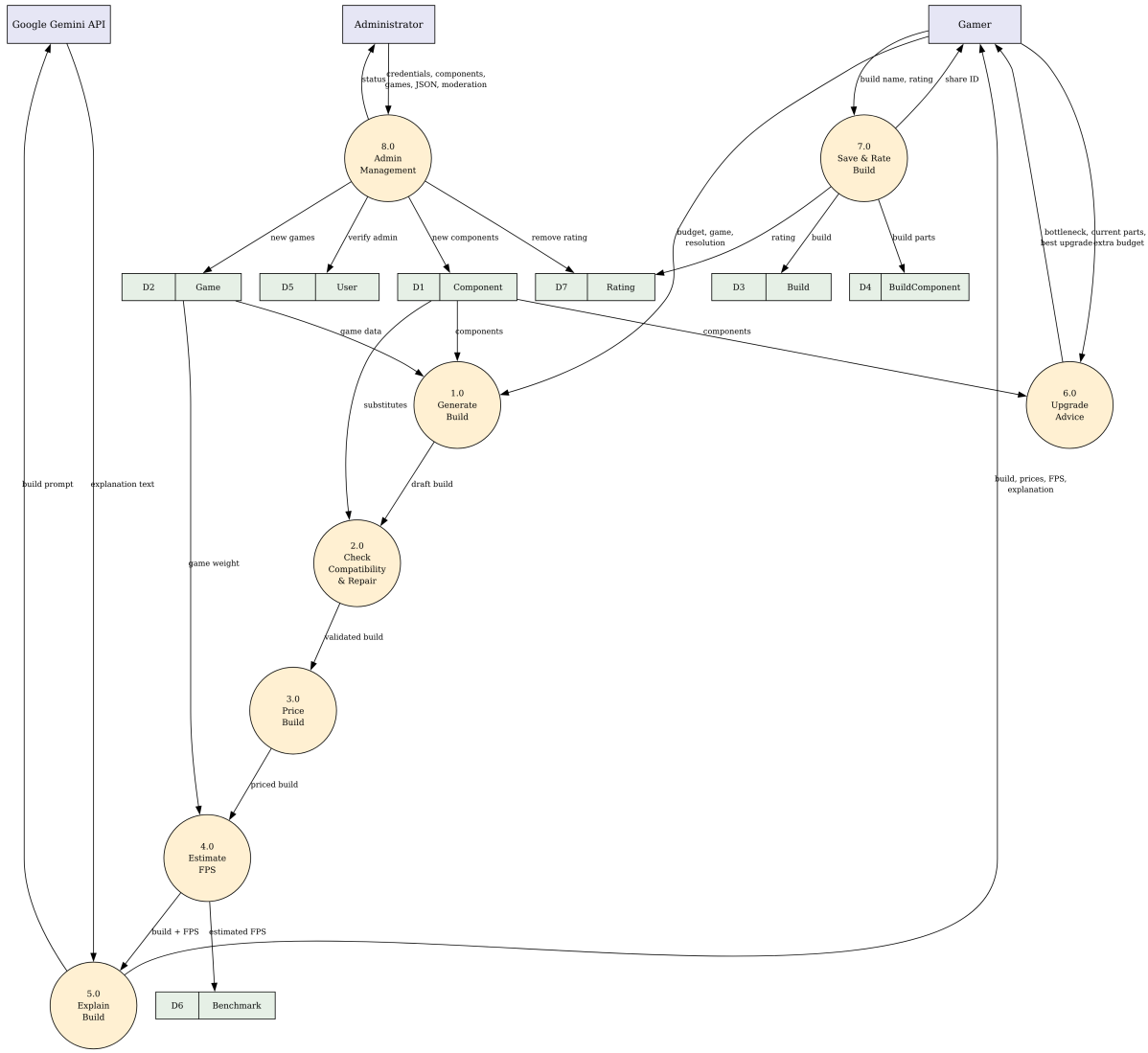
## 1. DFD Level 0 (Context Diagram)




Level 0 DFD poore BuildAdvisor system ko ek single process (0.0 BuildAdvisor System) ke roop mein dikhata hai, aur batata hai ki bahar ki entities system ke saath kaun sa data exchange karti hain.

| External Entity | System ko bhejta hai (Input) | System se milta hai (Output) |
|---|---|---|
| Gamer (User) | Budget, use-case/target game, resolution, preferences, login details, rating | Recommended build, price breakdown, FPS estimate, explanation, upgrade suggestions, saved builds |
| Administrator | Admin token, component/game/benchmark add-update-delete | Catalogue status, confirmation messages, ratings list |
| Gemini AI (External API) | Generated natural-language explanation | Build details prompt |

# 2. DFD Level 1

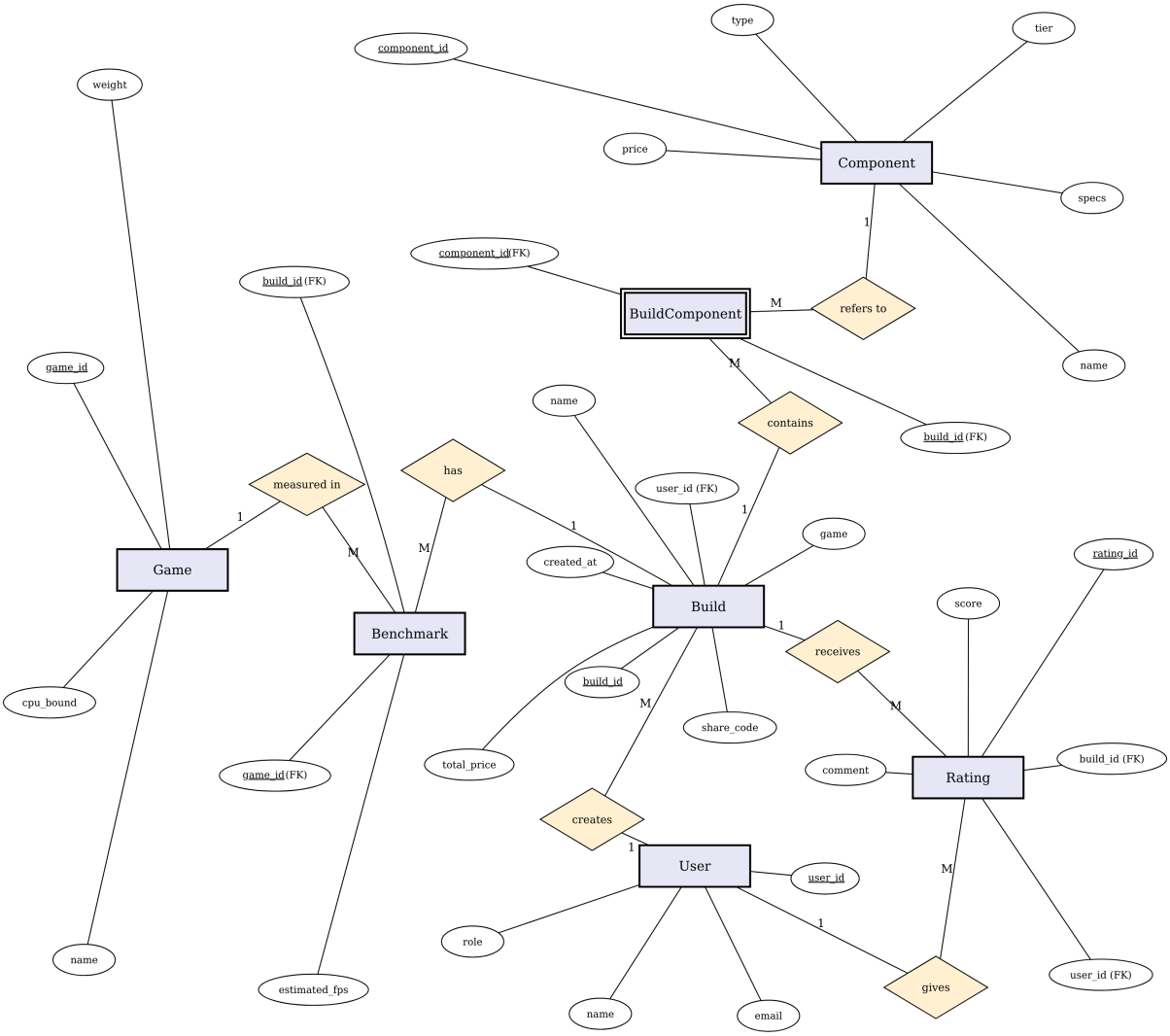



Level 1 mein system ko 8 major processes mein toda gaya hai, jo 7 data stores (D1–D7) se data padhte/likhte hain.

| Process | Kaam | Data Stores |
|---|---|---|
| 1.0 Generate Build | Budget ko tier ke hisaab se parts mein allocate karke best components choose karna | D1 Component, D2 Game |
| 2.0 Compatibility & Repair | Socket, RAM type, PSU wattage, case form-factor check; galat ho to repair (max 8 tries) | D1 Component |
| 3.0 Price Build | Har part ka price aur total compute karna | D1 Component |
| 4.0 Estimate FPS | Benchmark data se target game ka FPS estimate | D2 Game, D6 Benchmark |

| | | |
|---|---|---|
| 5.0 Explain Build | Gemini AI se explanation (fallback: template text) | Gemini (external) |
| 6.0 Upgrade Advisor | Existing build ke liye best upgrade suggest karna | D1, D3 |
| 7.0 Save & Rate | Build save karna aur user rating store karna | D3 Build, D4 BuildComponent, D5 User, D7 Rating |
| 8.0 Admin Management | Token verify karke catalogue manage karna | D1, D2, D6, D7 |

# 3. E-R Diagram




E-R diagram Chen notation mein database ka logical structure dikhata hai.

| Relationship | Type | Matlab |
|---|---|---|
| User – Build | 1 : M | Ek user kai builds save kar sakta hai |
| User – Rating | 1 : M | Ek user kai ratings de sakta hai |
| Build – Rating | 1 : M | Ek build ko kai ratings mil sakti hain |
| Build – Component | M : N (via BuildComponent) | Ek build mein kai components, ek component kai builds mein |
| Game – Component | M : N (via Benchmark) | Har game ke liye GPU/CPU ka FPS benchmark |

Foreign keys par **ON DELETE CASCADE** laga hai – user ya build delete hone par uske related records bhi automatically delete ho jaate hain.

# 4. Application Control Flow

Request layer-by-layer niche jaati hai:

```
app.py (Streamlit UI)
   │  HTTP request (JSON)
   ▼
api.py (Flask routes)
   │  validate input
   ▼
backend.py (orchestration)  ──► pricing.py (price_build)
   │                          ──► ai.py (Gemini explain_build)
   ▼
logic.py (generate_build, _repair, estimate_fps, upgrade)
   │
   ▼
db.py (SQLite — D1..D7)
```

## Generate Build – step by step

1. **Validate** – budget, resolution, game ko check karna; galat input par BackendError.
2. **Load catalogue** – db.py se components aur games load karna.
3. **generate_build** – budget tier select karke allocation: GPU 32%, CPU 20%, Motherboard 13%, RAM 10%, PSU 9%, Storage 8%, Case 8%.
4. **_repair** – compatibility issues ho to max 8 baar fix karne ki koshish.
5. **estimate_fps** – benchmark table se FPS nikalna.
6. **price_build** – final price breakdown.
7. **explain_build** – Gemini se explanation (fail ho to fallback text).
8. Result JSON wapas UI ko, jo cards/tables mein dikhaya jata hai.

## Other Flows

- **Save Build** – POST `/builds` → Build + BuildComponent mein insert.
- **Upgrade** – existing build + budget → best upgrade part suggest.
- **Admin** – har admin request par `X-Admin-Token` header verify; galat token par 401.
- **Error handling** – BackendError aane par UI mein red error chip dikhaya jata hai, app crash nahi hoti.